

Microsoft - 100 Technical Interview Questions

Microsoft (Software Engineer, New Grad)

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. Explain the concept of late binding vs early binding with an example.

Answer:

Early (static) binding resolves which method to call at compile time, based on the reference type - used for overloaded and static methods. Late (dynamic) binding resolves the method call at runtime, based on the actual object type - used for overridden instance methods, which is the basis of runtime polymorphism.

Q2. What is the difference between method overloading and method overriding? Give a code example of each.

Answer:

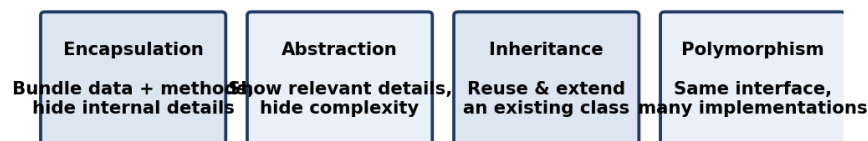
Overloading means multiple methods with the same name but different parameter lists in the same class (resolved at compile time), e.g. `add(int,int)` and `add(double,double)`. Overriding means a subclass provides its own implementation of a method already defined in its parent with the same signature (resolved at runtime), e.g. `Dog` overriding `Animal's makeSound()`.

Q3. Explain the four pillars of Object-Oriented Programming with a real-world example for each.

Answer:

The four pillars are: **Encapsulation** - bundling data and methods together and hiding internal state (e.g. a bank account class exposing `deposit()/withdraw()` but hiding the raw balance field); **Abstraction** - exposing only relevant details, like a car's steering wheel hiding the mechanics underneath; **Inheritance** - a Manager class reusing and extending an Employee class; **Polymorphism** - a `draw()` method behaving differently for a Circle vs a Square object called through a common Shape reference.

Four Pillars of OOP



Q4. What is an object in OOP, and how is it different from a class?

Answer:

An object is a concrete instance of a class, with its own state (field values) stored in memory, while a class is the blueprint/template that defines what fields and methods its objects will have. 'Car' is the class; 'myRedCar' is an object of that class.

Q5. What is method hiding, and how is it different from method overriding?

Answer:

Method hiding occurs when a subclass defines a static method with the same signature as a static method in its parent - the parent's version isn't overridden, just hidden, and which one runs depends on the reference type used to call it (resolved at compile time). Method overriding, in contrast, applies to instance methods and is resolved at runtime based on the actual object type.

Q6. Describe a scenario from your own project where you used inheritance, and explain why it was the right choice.

Answer:

Example answer structure: describe a project where a base class captured shared behaviour (e.g. a base Notification class with a send() method) and subclasses (EmailNotification, SMSNotification) overrode send() with channel-specific logic - explain why this avoided duplicating shared validation/logging code and made adding a new channel a matter of adding one new subclass.

Q7. What design pattern would you use to ensure only a single instance of a class exists, and how would you implement it thread-safely?

Answer:

The Singleton design pattern ensures only one instance of a class exists. A thread-safe implementation typically uses either double-checked locking with a volatile instance field, an eagerly-initialized static final instance, or (in Java) a single-element enum, all of which avoid two threads creating two separate instances under concurrent access.

Q8. Explain the Open/Closed Principle with a practical example.

Answer:

The Open/Closed Principle says classes should be open for extension but closed for modification. Practical example: instead of adding an if-else chain to a PaymentProcessor class every time a new payment method is added, define a PaymentMethod interface and add new payment types as new classes implementing it - existing code doesn't need to change.

DBMS & SQL

Q9. Write a SQL query using GROUP BY and HAVING to find departments with more than 5 employees.

Answer:

SELECT department, COUNT(*) AS emp_count FROM Employee GROUP BY department HAVING COUNT(*) > 5. GROUP BY collapses rows into per-department groups, and HAVING filters those groups (after aggregation) to keep only departments with more than five employees.

Q10. Explain how you would design a database schema for a simple e-commerce order system (tables and relationships).

Answer:

A minimal e-commerce schema would include: Customers (customer_id PK, name, email), Products (product_id PK, name, price), Orders (order_id PK, customer_id FK, order_date, status), and OrderItems (order_item_id PK, order_id FK, product_id FK, quantity, price_at_purchase) as a junction table capturing the many-to-many relationship between Orders and Products.

Q11. What is indexing, and what are the trade-offs of adding too many indexes to a table?

Answer:

An index is a data structure (commonly a B-Tree) that lets the database find rows matching a condition without scanning the whole table, dramatically speeding up SELECT queries. The trade-off is that every INSERT/UPDATE/DELETE must also update all indexes on that table, slowing down writes and consuming extra storage - so indexes should be added selectively on columns that are frequently filtered or joined on.

Q12. What is a view in SQL, and how is it different from a materialized view?

Answer:

A view is a virtual table defined by a stored SELECT query - it has no data of its own and is recomputed each time it's queried. A materialized view physically stores the result set on disk and must be refreshed periodically (or on trigger), trading storage and staleness for much faster read access compared to recomputing a complex query every time.

Q13. Explain database isolation levels and the anomalies (dirty read, non-repeatable read, phantom read) each one prevents.

Answer:

Isolation levels control how much one transaction can see of another's uncommitted work: Read Uncommitted allows dirty reads (seeing uncommitted changes); Read Committed prevents dirty reads but allows non-repeatable reads (a value changes between two reads in the same transaction); Repeatable Read prevents that but allows phantom reads (new rows appearing in a repeated range query); Serializable prevents all three by fully isolating transactions, at the cost of concurrency.

Q14. What is a stored procedure, and what are its advantages over writing ad-hoc queries?

Answer:

A stored procedure is a precompiled, named collection of SQL statements stored in the database and invoked as a single call. Advantages include reduced network round-trips (one call instead of many statements), better security (grant execute rights without exposing underlying tables directly), reusability across applications, and a cached execution plan that can improve performance over repeatedly sent ad-hoc queries.

Q15. Explain the difference between a primary key, a unique key, and a candidate key.

Answer:

A candidate key is any column (or set of columns) that could uniquely identify a row - a table can have several. A primary key is the one candidate key chosen to be the table's main unique identifier (cannot be NULL). A unique key is a constraint enforcing uniqueness on a column that is not the primary key, and unlike a primary key, it typically allows one NULL value.

Operating Systems

Q16. What is the difference between multiprogramming, multitasking, and multiprocessing?

Answer:

Multiprogramming keeps multiple programs in memory so the CPU always has something to execute when one process is waiting on I/O, improving utilization on a single CPU. Multitasking extends this idea to give the illusion of several programs running 'simultaneously' via time-slicing, often in an interactive context. Multiprocessing refers to a system with multiple physical CPUs/cores actually executing instructions in parallel at the same instant.

Q17. Explain what an inode is and what information it stores.

Answer:

An inode is a data structure in Unix-like file systems that stores a file's metadata: permissions, owner, size, timestamps, and pointers to the disk blocks holding the actual file data - notably, it does not store the file's name (that mapping lives in the containing directory), which is why hard links to the same inode can have different names.

Q18. Describe a real-world analogy to explain deadlock to a non-technical person.

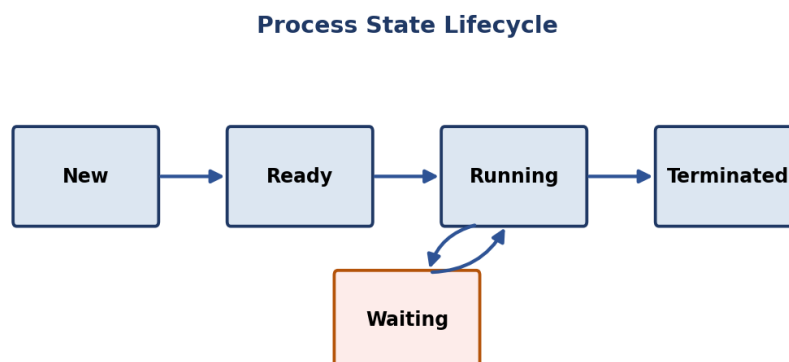
Answer:

Analogy: imagine four cars at a four-way intersection, each waiting for the car to its right to move first before it can proceed - if all four wait simultaneously, none will ever move, even though each car is only blocked because of the others' identical behavior. That's a deadlock: everyone is waiting for someone else to act first, and no one ever will.

Q19. What is the difference between a process and a thread?

Answer:

A process is an independent program in execution with its own memory space (code, data, heap, stack) and OS-allocated resources. A thread is a lightweight unit of execution within a process that shares the process's memory and resources with other threads of the same process, making thread creation and context switching cheaper than process creation.

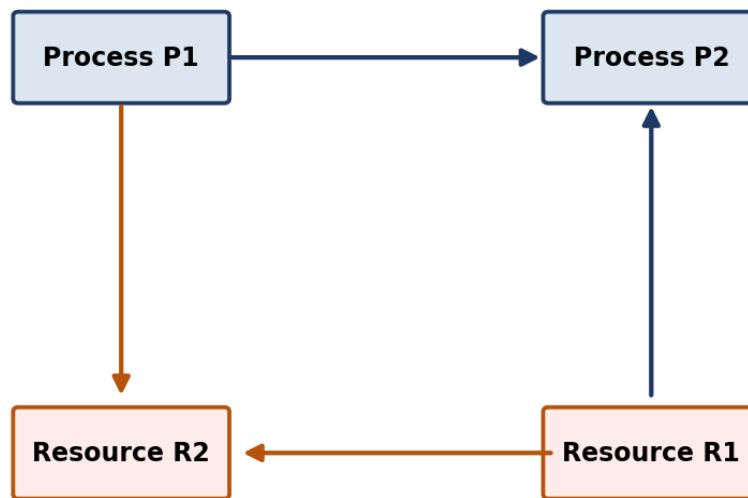


Q20. Explain the four necessary conditions for a deadlock to occur.

Answer:

The four necessary conditions are: **Mutual Exclusion** - at least one resource is held in a non-shareable mode; **Hold and Wait** - a process holds one resource while waiting for another; **No Preemption** - resources can't be forcibly taken away, only released voluntarily; **Circular Wait** - a closed chain of processes each waiting for a resource held by the next. All four must hold simultaneously for a deadlock to occur.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q21. What is spooling, and how does it help manage I/O devices like printers?

Answer:

Spooling (Simultaneous Peripheral Operations On-Line) buffers data intended for a slow I/O device (like a printer) on disk, letting a process 'print' instantly to the spool without waiting for the physical device, while a separate spooler process feeds jobs to the device at its own pace - allowing multiple processes to share one device without conflict.

Q22. How does an operating system ensure fair CPU allocation among multiple processes?

Answer:

The OS ensures fair CPU allocation through its scheduler, using algorithms like Round Robin (equal time slices for each process) or priority-based scheduling with aging (gradually increasing the priority of processes that have waited a long time, to prevent starvation), combined with preemption so no single process can monopolize the CPU indefinitely.

Q23. Explain how the Least Recently Used (LRU) page replacement algorithm works, and how you might implement it efficiently.

Answer:

LRU (Least Recently Used) evicts the page that hasn't been used for the longest time when a new page must be loaded and memory is full, based on the assumption that recently used pages are likely to be used again soon. An efficient implementation uses a combination of a hashmap (for O(1) lookup of a page's location) and a doubly linked list ordered by recency, so both finding a page and moving it to the 'most recently used' end take O(1) time.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Computer Networks

Q24. Explain the difference between IPv4 and IPv6.

Answer:

IPv4 uses 32-bit addresses (about 4.3 billion possible addresses, written as four decimal octets like 192.168.1.1) and is running out of available addresses. IPv6 uses 128-bit addresses (an astronomically larger space, written in hexadecimal groups like 2001:0db8::1), and also simplifies header processing and has built-in support for features IPv4 needed extensions for, like auto-configuration.

Q25. What is a subnet mask, and how do you calculate the number of usable hosts in a subnet?

Answer:

A subnet mask defines which portion of an IP address is the network part and which is the host part (e.g. 255.255.255.0 means the first 24 bits are network, the last 8 are host). For a /24 subnet, there are $2^8 = 256$ total addresses, minus 2 reserved (network address and broadcast address), giving 254 usable host addresses.

Q26. What is the difference between HTTP and HTTPS, and how does TLS/SSL secure a connection?

Answer:

HTTP sends data in plaintext, so anyone intercepting the traffic can read it. HTTPS wraps HTTP inside a TLS/SSL layer that encrypts the data in transit. Under the hood, a TLS handshake occurs first: the client and server agree on a cipher suite, the server presents a certificate to prove its identity, and both sides derive a shared symmetric session key used to encrypt all subsequent HTTP traffic.

Q27. What is NAT, and why is it necessary given the limited IPv4 address space?

Answer:

NAT (Network Address Translation) allows multiple devices on a private network to share a single public IP address when communicating with the internet, translating private IPs to the public IP (and back for return traffic) at the router/gateway. It's necessary because IPv4's roughly 4.3 billion addresses are far fewer than the number of internet-connected devices worldwide.

Data Structures & Algorithms

Q28. What is Big-O notation, and why do we use asymptotic analysis instead of exact runtimes?

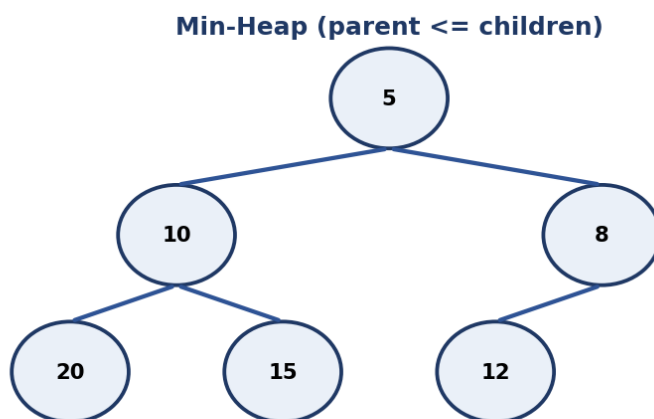
Answer:

Big-O notation describes the upper-bound growth rate of an algorithm's running time or space as input size approaches infinity, ignoring constant factors and lower-order terms. We use asymptotic analysis instead of exact runtimes because exact timings vary by hardware, language, and implementation details, while Big-O gives a hardware-independent way to compare how algorithms scale as input grows large.

Q29. Explain how a min-heap/max-heap is implemented and how it supports efficient priority queue operations.

Answer:

A heap is a complete binary tree (usually implemented as an array) satisfying the heap property: in a min-heap, every parent is less than or equal to its children (so the minimum is always at the root); in a max-heap, every parent is greater than or equal to its children. Insertion and extraction of the min/max both take $O(\log n)$ since you only need to 'bubble' an element up or down the height of the tree, making heaps the standard backing structure for an efficient priority queue.



Q30. Explain topological sorting and where it is used.

Answer:

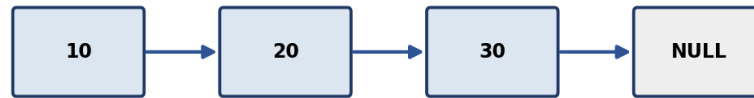
Topological sorting produces a linear ordering of a Directed Acyclic Graph's (DAG) vertices such that for every directed edge $u \rightarrow v$, u comes before v in the ordering. It's used wherever tasks have dependencies that must be respected, such as course prerequisite ordering, build systems compiling files in the right order, or task scheduling pipelines.

Q31. How would you detect a cycle in a linked list? Explain Floyd's cycle detection algorithm.

Answer:

Floyd's cycle detection (the 'tortoise and hare' algorithm) uses two pointers moving through the linked list at different speeds - one advances one node at a time (slow), the other advances two nodes at a time (fast). If there is a cycle, the fast pointer will eventually 'lap' the slow pointer and they will meet at the same node; if the fast pointer reaches the end (null), there is no cycle. This runs in $O(n)$ time and $O(1)$ space.

Singly Linked List



Q32. Explain the two-pointer technique with an example problem.

Answer:

The two-pointer technique uses two indices moving through a data structure (often from opposite ends or at different speeds) to avoid nested loops. Example: given a sorted array, to find a pair summing to a target, start one pointer at the beginning and one at the end - if the sum is too large, move the right pointer left; if too small, move the left pointer right - solving the problem in $O(n)$ instead of the $O(n^2)$ brute force of checking every pair.

Q33. What is backtracking, and how is it different from brute force?

Answer:

Backtracking systematically explores all candidate solutions, but abandons ('backtracks' from) a partial candidate as soon as it determines that candidate cannot possibly lead to a valid solution, rather than exploring it to completion. This pruning makes it far more efficient than pure brute force, which would fully explore every possibility regardless of early signs of failure - classic examples include the N-Queens problem and Sudoku solvers.

Q34. How would you find the kth largest element in an unsorted array efficiently?

Answer:

One efficient approach uses a min-heap of size k : iterate through the array, pushing elements onto the heap and popping the smallest whenever the heap exceeds size k - after processing all elements, the heap's root is the k th largest, achieving $O(n \log k)$ time. Alternatively, Quickselect (a partition-based approach related to Quicksort) finds it in average $O(n)$ time.

Q35. Explain how you would design an LRU cache and the data structures you'd use to achieve $O(1)$ get/put operations.

Answer:

An LRU cache needs $O(1)$ get and put operations. The standard design combines a hashmap (key $\rightarrow$ pointer to a node) for instant lookup with a doubly linked list ordered by recency of use: on access, the accessed node is unlinked and moved to the 'most recently used' end in $O(1)$; when the cache exceeds capacity, the node at the 'least recently used' end is evicted, also in $O(1)$, with the hashmap updated accordingly.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q36. How would you merge two sorted arrays/linked lists efficiently?

Answer:

Use a two-pointer merge: maintain one pointer into each sorted array/list, repeatedly compare the elements they point to, append the smaller one to the result, and advance that pointer - continue until one input is exhausted, then append the remainder of the other. This runs in $O(n + m)$ time, which is also the core merge step used inside Merge Sort.

Q37. Explain Dijkstra's algorithm for shortest paths and its limitations with negative edge weights.

Answer:

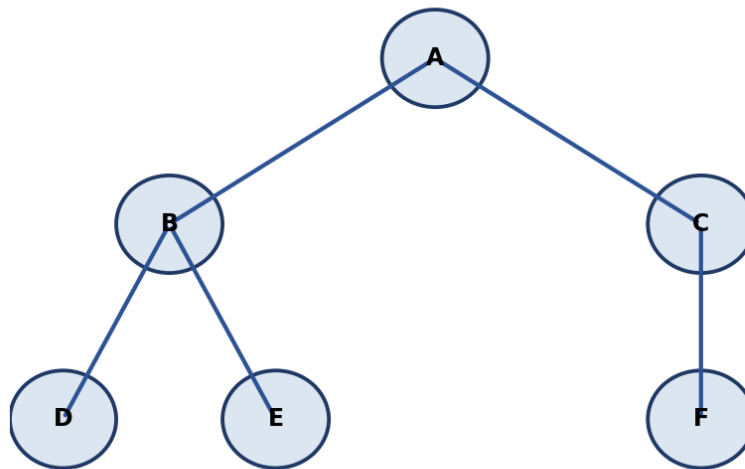
Dijkstra's algorithm finds the shortest path from a source node to all others in a weighted graph by repeatedly selecting the unvisited node with the smallest known distance, then 'relaxing' (updating) the distances of its neighbors - using a priority queue/min-heap, it runs in $O(E \log V)$. Its key limitation is that it assumes all edge weights are non-negative; a negative edge can invalidate its greedy assumption that once a node is finalized its distance can't improve, requiring an algorithm like Bellman-Ford instead.

Q38. Explain the difference between BFS and DFS graph traversal, including their use cases.

Answer:

BFS (Breadth-First Search) explores a graph level by level using a queue, visiting all neighbors of a node before moving deeper - useful for finding the shortest path in an unweighted graph. DFS (Depth-First Search) explores as far as possible along one branch before backtracking, using a stack (or recursion) - useful for tasks like detecting cycles, topological sorting, or exploring all possible paths/states.

Graph: BFS order A,B,C,D,E,F | DFS order A,B,D,E,C,F



Q39. Explain the difference between time complexity and space complexity with examples.

Answer:

Time complexity measures how an algorithm's runtime grows as input size increases (e.g. $O(n)$ for a single loop through n elements). Space complexity measures how much additional memory an algorithm needs relative to input size (e.g. $O(1)$ for an in-place swap vs $O(n)$ for creating a copy of the array). Both are usually expressed using Big-O notation and both matter when choosing an algorithm for constrained environments.

Q40. What is the sliding window technique, and in what type of problems is it useful?

Answer:

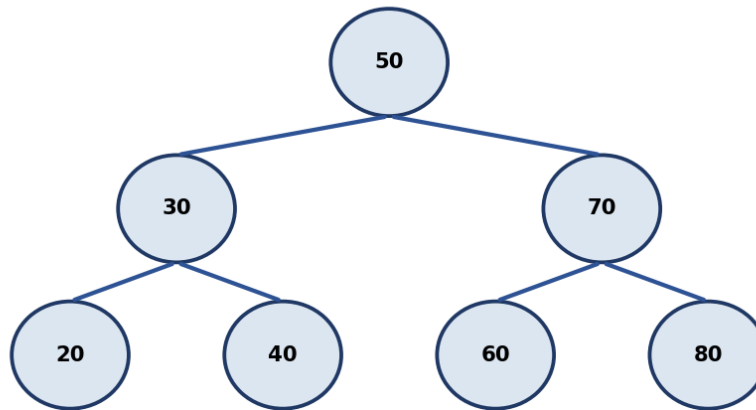
The sliding window technique maintains a contiguous subrange ('window') over an array or string and expands/shrinks its boundaries incrementally rather than recomputing from scratch for every possible subrange. It's especially useful for problems asking for the best/longest/shortest subarray or substring satisfying some condition (e.g. maximum sum subarray of size k , or longest substring without repeating characters), typically turning an $O(n^2)$ brute force into an $O(n)$ solution.

Q41. What is a binary search tree, and what is the time complexity of insertion, deletion, and search in the average and worst case?

Answer:

A Binary Search Tree (BST) is a tree where every node's left subtree contains only smaller values and its right subtree contains only larger values, enabling efficient ordered operations. In a balanced BST, insertion, deletion, and search are all $O(\log h)$ where h is the height (roughly $\log n$ for a balanced tree). In the worst case (a degenerate, list-like tree caused by inserting sorted data without rebalancing), all three operations degrade to $O(n)$.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)



Q42. What is the difference between a greedy algorithm and dynamic programming? Give an example problem for each.

Answer:

A greedy algorithm makes the locally optimal choice at each step without reconsidering past decisions, and works only for problems where local optimality leads to a global optimum (e.g. activity/interval selection, Huffman coding). Dynamic programming considers overlapping subproblems and combines their optimal solutions, guaranteeing a global optimum for a broader class of problems (e.g. the 0/1 Knapsack problem, where greedy choice-by-choice doesn't always give the best result).

Q43. Compare Merge Sort and Quick Sort - discuss best/worst case complexity, stability, and when you'd choose one over the other.

Answer:

Merge Sort has guaranteed $O(n \log n)$ time in best, average, and worst cases, is stable, but requires $O(n)$ extra space for merging. Quick Sort also averages $O(n \log n)$ but can degrade to $O(n^2)$ in the worst case (e.g. already-sorted input with a poor pivot choice), is typically not stable, but sorts in-place with better real-world constant factors and cache performance. Choose Merge Sort when stability or guaranteed worst-case performance matters (e.g. external sorting); choose Quick Sort for general in-memory sorting where average-case speed matters more.

Sorting Algorithm Complexity Comparison

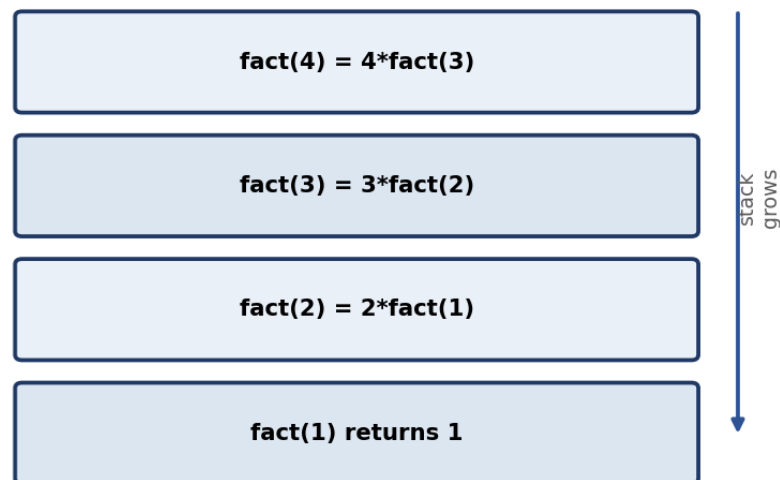
Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q44. Explain how recursion uses the call stack internally, and what causes a stack overflow.

Answer:

Each recursive call pushes a new stack frame onto the call stack, containing that call's local variables, parameters, and return address. As calls nest deeper, more frames stack up; when a call returns, its frame is popped off. If recursion goes too deep (e.g. no base case, or a base case never reached due to a bug), the call stack exceeds its allocated size, causing a stack overflow.

Call Stack while computing factorial(4)



Q45. Explain how binary search works and derive its time complexity.

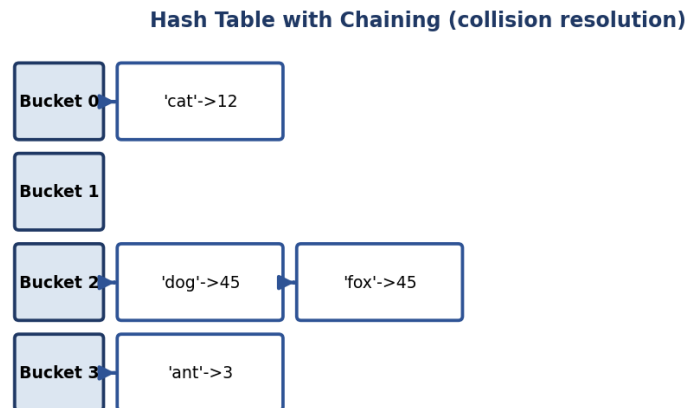
Answer:

Binary search repeatedly divides a sorted array in half: compare the target to the middle element, and discard the half that can't contain the target, narrowing the search range each time. Since the search space is halved every step, the time complexity is $O(\log n)$ - a huge improvement over linear search's $O(n)$ for large sorted datasets.

Q46. Explain how a hash table works internally, including collision resolution strategies.

Answer:

A hash table maps keys to array indices using a hash function, giving average $O(1)$ time for insert/search/delete. Since different keys can hash to the same index (a collision), collisions are typically resolved via chaining (each bucket holds a linked list/array of entries that hashed there) or open addressing (probing for the next free slot in the array itself, e.g. linear or quadratic probing).

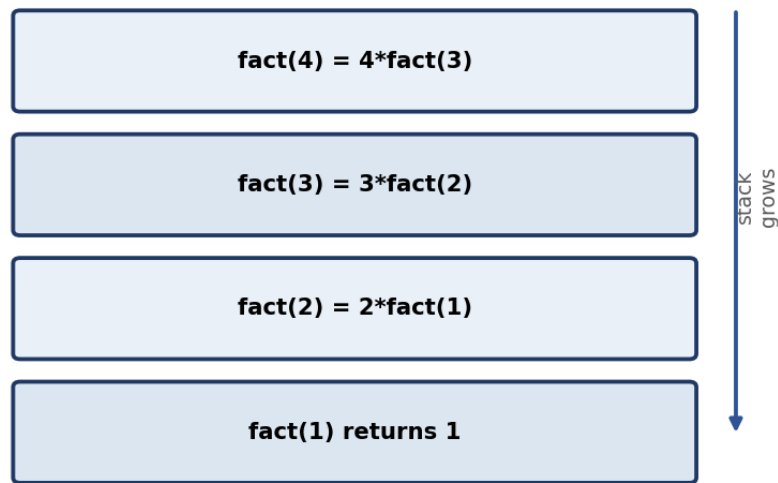


Q47. What is dynamic programming, and how does it differ from plain recursion and from greedy algorithms?

Answer:

Dynamic programming solves problems by breaking them into overlapping subproblems, solving each just once, and storing (memoizing) the results to avoid redundant recomputation - it's applicable when a problem has both optimal substructure and overlapping subproblems. Plain recursion without memoization re-solves the same subproblems repeatedly, often leading to exponential time. Unlike a greedy algorithm (which makes one irrevocable locally-optimal choice at each step), DP considers combinations of subproblem solutions to guarantee a globally optimal result when applicable.

Call Stack while computing factorial(4)

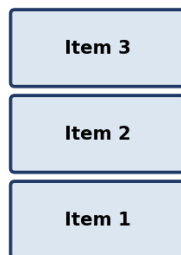


Q48. What is the difference between a stack and a queue, and where is each used in real systems?

Answer:

A stack follows Last-In-First-Out (LIFO) order - used for undo functionality, expression evaluation, and the call stack behind recursion. A queue follows First-In-First-Out (FIFO) order - used for task scheduling, breadth-first search, and handling requests in the order they arrive (e.g. print queues, message queues).

Stack (LIFO)



<- Push / Pop (top)

Queue (FIFO)



Enqueue ->

Q49. Explain amortized time complexity using the example of a dynamic array's append operation.

Answer:

Amortized time complexity averages the cost of an operation over a sequence of operations, even if occasional individual operations are expensive. A dynamic array's append is usually $O(1)$, but when the underlying fixed-size array is full, it must allocate a new (typically double-sized) array and copy all existing elements - an $O(n)$ operation. Because this expensive resize happens increasingly rarely as the array grows, the average cost per append across many operations still works out to $O(1)$ amortized.

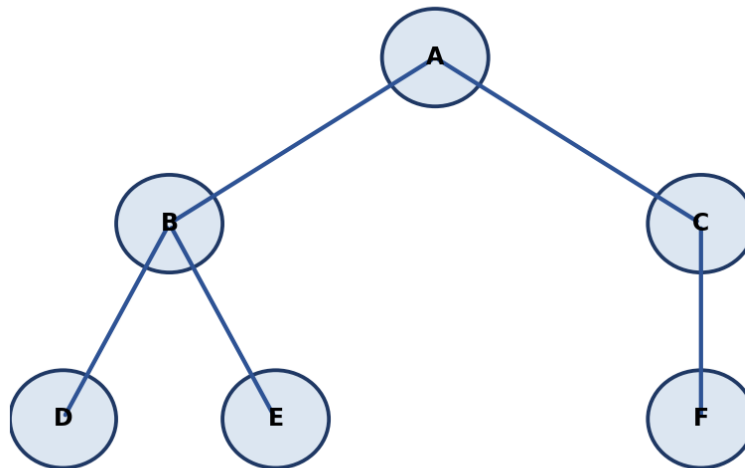
Coding Problems

Q50. Write a program to count the number of islands in a 2D grid (graph/matrix traversal).

Answer:

Treat the grid as an implicit graph where each land cell ('1') is a node connected to its up/down/left/right land neighbors. Scan every cell; whenever an unvisited land cell is found, increment the island count and run a BFS or DFS from that cell to mark every connected land cell as visited, so it isn't counted again. This runs in $O(\text{rows} * \text{cols})$ time since each cell is visited a constant number of times.

Graph: BFS order A,B,C,D,E,F | DFS order A,B,D,E,C,F

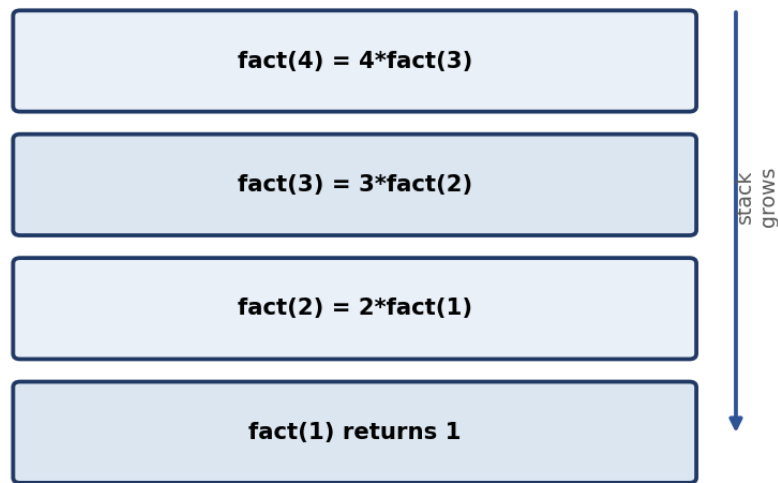


Q51. Write a program to find the factorial of a number using recursion.

Answer:

Recursive definition: $\text{factorial}(n) = n * \text{factorial}(n-1)$, with a base case $\text{factorial}(0) = 1$ (or $\text{factorial}(1) = 1$). Each call multiplies n by the result of the smaller subproblem until the base case is reached, then the multiplications unwind back up the call stack. Time complexity is $O(n)$; space complexity is $O(n)$ due to the recursive call stack.

Call Stack while computing factorial(4)



Q52. Write a program to merge overlapping intervals given a list of intervals.

Answer:

Sort the intervals by their start time, then iterate through them while maintaining a 'current merged interval' - if the next interval's start is less than or equal to the current merged interval's end, merge them by extending the end to the maximum of the two ends; otherwise, push the current merged interval to the result and start a new one with the next interval. This runs in $O(n \log n)$ time due to the initial sort.

Q53. Write a program to check whether a given string is a palindrome.

Answer:

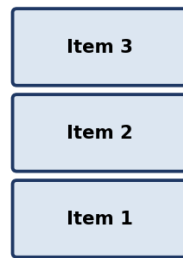
Use two pointers, one from the start (left) and one from the end (right) of the string, comparing characters and moving both pointers inward - if any pair doesn't match, it's not a palindrome; if the pointers meet or cross without mismatches, it is. This runs in $O(n)$ time and $O(1)$ extra space (ignoring the input itself).

Q54. Write a program to implement a stack using two queues (or vice versa).

Answer:

To implement a queue using two stacks: push new elements onto stack1; for dequeue, if stack2 is empty, pop everything from stack1 and push it onto stack2 (reversing the order), then pop from stack2 - this gives FIFO behavior with amortized $O(1)$ per operation. Implementing a stack using two queues works similarly in reverse, rearranging elements between the two queues so the most recently added element comes out first.

Stack (LIFO)



<- Push / Pop (top)

Queue (FIFO)



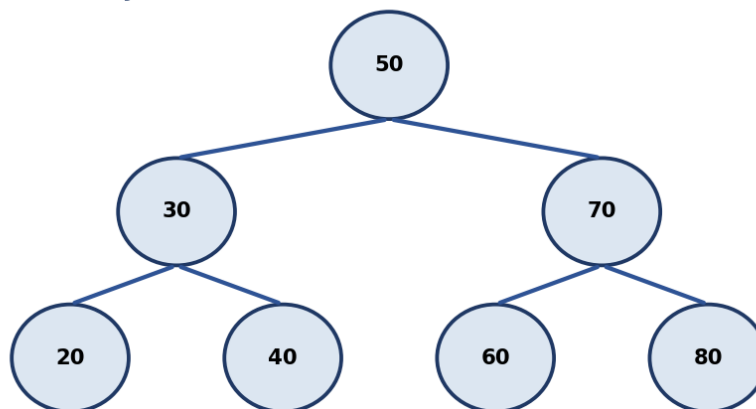
Enqueue ->

Q55. Write a program to find the lowest common ancestor (LCA) of two nodes in a binary tree.

Answer:

For a BST, the LCA can be found by starting at the root and comparing both target node values to the current node's value: if both are smaller, move to the left child; if both are larger, move to the right child; if they diverge (one smaller, one larger, or one equals the current node), the current node is the LCA. This runs in $O(h)$ time where h is the tree height. For a general binary tree (not necessarily a BST), a recursive post-order search that returns the first node where both targets are found in different subtrees is used instead.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)



Q56. Write a program to find the maximum subarray sum (Kadane's Algorithm).

Answer:

Kadane's Algorithm scans the array once, maintaining a running 'current sum' that resets to the current element whenever adding the previous running sum would make it smaller than the element alone (i.e. $\text{current_sum} = \max(\text{arr}[i], \text{current_sum} + \text{arr}[i])$), while tracking the overall maximum seen so far. This finds the maximum subarray sum in $O(n)$ time and $O(1)$ space.

Q57. Write a program to determine if a given number is prime, and optimize it for large inputs.

Answer:

Check divisibility only up to the square root of the number (if n has a factor larger than $\sqrt{n}$, it must also have a corresponding factor smaller than $\sqrt{n}$), skipping even numbers after checking 2, which reduces the check to roughly $O(\sqrt{n})$. For checking primality of many numbers up to a limit, the Sieve of Eratosthenes precomputes all primes up to that limit in $O(n \log \log n)$, which is far more efficient than testing each number individually.

Q58. Write a program to find the first non-repeating character in a string.

Answer:

Use a hashmap to count the frequency of each character in a single pass through the string, then iterate through the string again in order and return the first character whose count is exactly 1. This runs in $O(n)$ time and $O(k)$ space, where k is the size of the character set.

Q59. Write a program to implement a basic LRU cache using a hashmap and a doubly linked list.

Answer:

Maintain a hashmap from key to a reference to its node in a doubly linked list ordered by recency of use. On `get(key)`: if present, move that node to the front (most recently used end) and return its value, else return `-1/None`. On `put(key, value)`: if the key exists, update its value and move it to the front; otherwise insert a new node at the front, and if capacity is exceeded, remove the node at the back of the list (least recently used) along with its hashmap entry. Both operations run in $O(1)$.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q60. Write a program to implement binary search on a rotated sorted array.

Answer:

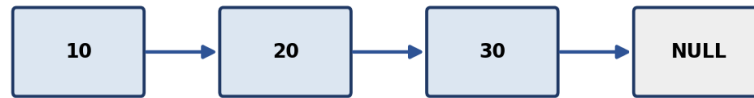
Modify binary search by first determining which half of the array (from mid to low, or mid to high) is properly sorted, then checking if the target lies within that sorted half's range - if so, search recurse/iterate into that half, otherwise search the other half. This preserves the $O(\log n)$ time complexity of standard binary search despite the rotation.

Q61. Write a program to reverse a singly linked list (iterative and recursive approaches).

Answer:

Iterative: maintain a 'previous' pointer (initially null) and a 'current' pointer (initially head); at each step, save `current.next`, point `current.next` to previous, then advance previous and current forward - repeat until current is null, then previous is the new head. Recursive: recurse to the end of the list first, then reverse the pointer of each node on the way back up (`node.next.next = node`; `node.next = null`). Both run in $O(n)$ time; iterative uses $O(1)$ extra space while recursive uses $O(n)$ call-stack space.

Singly Linked List



Q62. Write a program to implement a queue using a fixed-size array (circular queue).

Answer:

A circular queue uses a fixed-size array with 'front' and 'rear' indices that wrap around using modulo arithmetic ($\text{rear} = (\text{rear} + 1) \% \text{capacity}$) once they reach the end of the array, allowing the queue to reuse freed space at the beginning after elements have been dequeued - avoiding the wasted space of a naive fixed-array queue that never reuses earlier slots.

Q63. Write a program to rotate an array by k positions.

Answer:

Reverse the whole array, then reverse the first k elements, then reverse the remaining n-k elements - this three-step reversal rotates the array by k positions in-place in $O(n)$ time and $O(1)$ extra space. (Alternatively, use an auxiliary array of size n to place each element directly at its rotated index, trading $O(n)$ space for simpler code.)

Q64. Write a program to find the middle element of a linked list in a single pass.

Answer:

Use the 'slow and fast pointer' technique: the slow pointer advances one node at a time while the fast pointer advances two nodes at a time - when the fast pointer reaches the end of the list, the slow pointer will be exactly at the middle. This finds the middle node in a single pass, in $O(n)$ time and $O(1)$ space.

Q65. Write a program to find the missing number in an array containing n distinct numbers from 0 to n.

Answer:

Sum formula approach: the expected sum of numbers 0 to n is $n*(n+1)/2$; subtract the actual sum of the given array from this expected sum, and the difference is the missing number - $O(n)$ time, $O(1)$ space. Alternatively, XOR every number from 0 to n together with every element in the array; since XOR-ing a number with itself cancels out to zero, all present numbers cancel, leaving only the missing number.

Q66. Write a program to remove duplicates from a sorted array in-place.

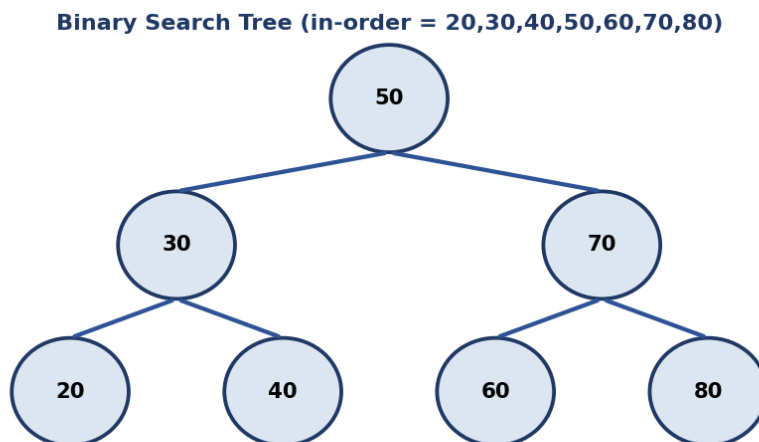
Answer:

Since the array is already sorted, use a slow pointer marking the last position of a unique element written so far, and a fast pointer scanning ahead - whenever the fast pointer finds a value different from the one at the slow pointer, advance the slow pointer and copy that new value into place. This removes duplicates in-place in $O(n)$ time and $O(1)$ extra space.

Q67. Write a program to check if a given binary tree is a valid Binary Search Tree.

Answer:

Perform an in-order traversal of the tree and check that the resulting sequence of values is strictly increasing - a valid BST always produces a sorted sequence under in-order traversal. Alternatively, recursively validate each node against a running (min, max) valid range inherited from its ancestors, narrowing the range as you descend left or right - both approaches run in $O(n)$ time.



Q68. Write a program to flatten a nested list/array.

Answer:

Use recursion: iterate through each element of the (possibly nested) list - if an element is itself a list, recursively flatten it and extend the result with its contents; if it's a plain value, append it directly to the result list. This naturally handles arbitrary nesting depth, with time complexity $O(n)$ where n is the total number of elements across all nesting levels.

Q69. Write a program to find all pairs in an array that sum up to a given target value.

Answer:

Use a hashmap: iterate through the array once, and for each element x , check if $(\text{target} - x)$ already exists in the hashmap - if so, record the pair; otherwise, add x to the hashmap and continue. This finds all pairs summing to the target in $O(n)$ time and $O(n)$ space, instead of the $O(n^2)$ brute-force nested loop.

Q70. Write a program to find the longest common subsequence between two strings.

Answer:

Use dynamic programming: build a 2D table $dp[i][j]$ representing the length of the longest common subsequence between the first i characters of string A and first j characters of string B. If $A[i-1] == B[j-1]$, $dp[i][j] = dp[i-1][j-1] + 1$; otherwise $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$. This runs in $O(n*m)$ time and space, where n and m are the string lengths.

Q71. Write a program to check if two strings are anagrams of each other.

Answer:

Two common approaches: (1) sort both strings and check if the sorted versions are equal ($O(n \log n)$); or (2) build a frequency count (e.g. a 26-length array for lowercase letters, or a hashmap for general characters) of one string, then decrement counts while scanning the second string - if all counts return to zero and lengths match, they're anagrams ($O(n)$ time, $O(1)$ or $O(k)$ space).

Q72. Write a program to find the intersection of two arrays.

Answer:

Efficient approach: put all elements of the smaller array into a hash set for $O(1)$ lookup, then iterate through the other array and collect any element found in the set (adding it to a result set to avoid duplicates in the output). This runs in $O(n + m)$ time and $O(\min(n, m))$ space.

Q73. Write a program to check whether a binary tree is balanced.

Answer:

Recursively compute the height of the left and right subtrees for every node; if at any point the difference in height between left and right subtrees exceeds 1 for any node, the tree is unbalanced. An efficient implementation computes height and checks balance in the same bottom-up traversal (returning -1 as a sentinel to signal 'unbalanced' early), achieving $O(n)$ time instead of the naive $O(n^2)$.

Programming Language Fundamentals

Q74. Explain the difference between an abstract class and a concrete class with an example from a language you know well.

Answer:

An abstract class can declare abstract methods (no body, must be implemented by subclasses) alongside concrete methods with actual implementations, and cannot be instantiated directly - e.g. an abstract Shape class with an abstract area() method but a concrete describe() method common to all shapes. A concrete class provides full implementations for all its methods and can be instantiated directly, e.g. a Circle class extending Shape and implementing area().

Q75. What are lambda expressions/anonymous functions, and when would you use them?

Answer:

A lambda expression (or anonymous function) is a small, unnamed function defined inline, typically used for short operations passed as arguments to other functions - e.g. sorting a list with a custom key: sorted(items, key=lambda x: x.age). They're best used for simple, one-off logic; anything more complex or reused in multiple places is usually clearer as a named function.

Q76. What is exception handling, and why is it important to avoid empty catch blocks?

Answer:

Exception handling is the mechanism for detecting and responding to runtime errors (like a missing file or invalid input) gracefully, using constructs like try/catch/finally, instead of letting the program crash. Empty catch blocks are dangerous because they silently swallow errors - the program continues running as if nothing went wrong, making it far harder to diagnose bugs later since there's no log, message, or fallback behavior indicating that something actually failed.

Q77. Explain the difference between pass-by-value and pass-by-reference with an example.

Answer:

Pass-by-value copies the actual value of a variable into the function's parameter, so changes inside the function don't affect the original variable. Pass-by-reference passes a reference to the original variable's memory location, so changes inside the function do affect the original. Example: in a language with true pass-by-reference, `swap(a, b)` can actually swap the caller's variables; with pass-by-value, it would only swap local copies, leaving the caller's variables unchanged.

Q78. What is garbage collection, and how does it help prevent memory leaks?

Answer:

Garbage collection is an automatic memory management process that identifies objects no longer reachable/referenced by the running program and reclaims their memory, so the developer doesn't have to manually free it. This helps prevent memory leaks caused by forgetting to deallocate memory, though a program can still 'leak' memory logically if it keeps unnecessary references alive (e.g. objects sitting in a growing cache that's never cleared).

Q79. Explain the difference between static and dynamic memory allocation.

Answer:

Static memory allocation reserves a fixed amount of memory at compile time (e.g. a fixed-size array or a local variable on the stack), which is fast but inflexible since the size must be known ahead of time. Dynamic memory allocation reserves memory at runtime (e.g. via `malloc/new`), allowing the program to request exactly the amount of memory it needs based on runtime conditions, at the cost of manual (or garbage-collected) management and slightly slower access.

Q80. What is the difference between a compiler and an interpreter?

Answer:

A compiler translates the entire source program into machine code (or another target language) ahead of time, producing an executable that runs independently afterward - errors are caught before execution, and execution tends to be faster since translation work is already done. An interpreter reads and executes source code line-by-line (or statement-by-statement) at runtime without producing a standalone executable, which makes development/debugging faster but execution generally slower since translation happens repeatedly during each run.

Q81. What is method chaining, and how would you implement it in a class you design?

Answer:

Method chaining lets you call multiple methods on the same object in a single statement, e.g. `builder.setName('X').setAge(30).build()`. To implement it, each method that would normally return void instead returns 'this' (the current object instance), allowing the next method call to be appended directly onto the previous one's result - commonly used in builder-pattern classes for constructing complex objects readably.

Q82. Explain the concept of multithreading vs multiprocessing, and when you would choose one over the other.

Answer:

Multithreading runs multiple threads within a single process, sharing memory - lightweight and fast to communicate between threads, but requires careful synchronization to avoid race conditions, and (in languages like Python with a GIL) doesn't achieve true CPU parallelism. Multiprocessing runs separate processes, each with its own memory space - avoids shared-memory race conditions and achieves true parallelism across CPU cores, but has higher overhead for creation and inter-process communication. Choose multithreading for I/O-bound work and multiprocessing for CPU-bound work in GIL-constrained languages.

Web & Cloud Fundamentals

Q83. What is auto-scaling in cloud computing, and what metrics typically trigger it?

Answer:

Auto-scaling automatically adjusts the number of running compute instances up or down based on real-time demand, rather than requiring manual intervention. Common triggering metrics include CPU utilization, memory usage, request/queue length, and response latency - when a threshold is crossed for a sustained period, the auto-scaler adds or removes instances accordingly.

Q84. Explain the concept of statelessness in REST APIs and why it matters for scalability.

Answer:

Statelessness means each request from a client to a server must contain all the information the server needs to process it, with no reliance on server-side session state stored between requests. This matters for scalability because any server instance can handle any request without needing shared session state, making it trivial to add more server instances behind a load balancer without worrying about which server 'remembers' a given client.

Q85. What is a RESTful API, and what makes an API design 'RESTful'?

Answer:

A RESTful API is an API designed around REST (Representational State Transfer) architectural principles: it uses standard HTTP methods (GET, POST, PUT, DELETE) mapped to resource operations, is stateless (each request contains all information needed), uses resource-based URLs (e.g. /users/123), and typically returns representations of resources (often JSON). An API is 'RESTful' to the extent it actually follows these constraints rather than just using HTTP as a transport.

Q86. What is the difference between HTTP and HTTPS, and how is HTTPS implemented under the hood?

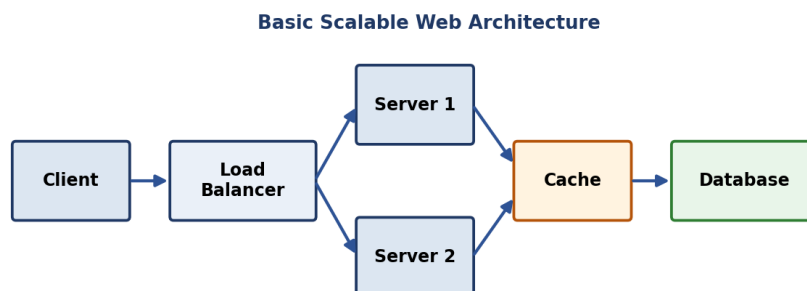
Answer:

HTTP transmits data in plaintext, so it's vulnerable to eavesdropping and tampering in transit. HTTPS secures HTTP by layering it on top of TLS/SSL: a handshake negotiates a shared session key and verifies the server's identity via its certificate, and all subsequent HTTP traffic is encrypted using that session key, protecting confidentiality and integrity even over an untrusted network.

Q87. What is the purpose of an API Gateway in a microservices-based application?

Answer:

An API Gateway sits in front of a microservices architecture as the single entry point for client requests, routing each request to the appropriate backend service. It commonly also handles cross-cutting concerns centrally - authentication/authorization, rate limiting, request/response transformation, and logging - so individual microservices don't each need to reimplement that logic.



Q88. Explain how a message queue like Kafka or RabbitMQ helps decouple microservices.

Answer:

A message queue like Kafka or RabbitMQ sits between producer services (that generate events/messages) and consumer services (that process them), letting producers publish messages without waiting for consumers to be ready or fast. This decouples services in time and load - producers and consumers can scale, fail, or be deployed independently - and provides buffering so a burst of traffic doesn't overwhelm downstream consumers directly.

Q89. Explain the difference between IaaS, PaaS, and SaaS with real examples of each.

Answer:

IaaS (Infrastructure as a Service) provides raw virtualized compute/storage/networking that you configure yourself, e.g. AWS EC2 or Azure VMs. PaaS (Platform as a Service) provides a managed platform to build and deploy applications without managing the underlying servers, e.g. Heroku or Azure App Service. SaaS (Software as a Service) delivers a complete, ready-to-use application over the internet, e.g. Gmail or Salesforce - you just use it, with no infrastructure or platform management at all.

Q90. Explain how a CDN improves website performance for geographically distributed users.

Answer:

A CDN caches static content (images, CSS, JS, video) on edge servers distributed across many geographic locations. When a user requests that content, it's served from the nearest edge server instead of traveling all the way to the origin server, reducing round-trip latency, offloading traffic from the origin, and improving load times especially for users far from the origin's data center.

System Design Basics

Q91. How would you design a basic caching layer for a high-traffic read-heavy application?

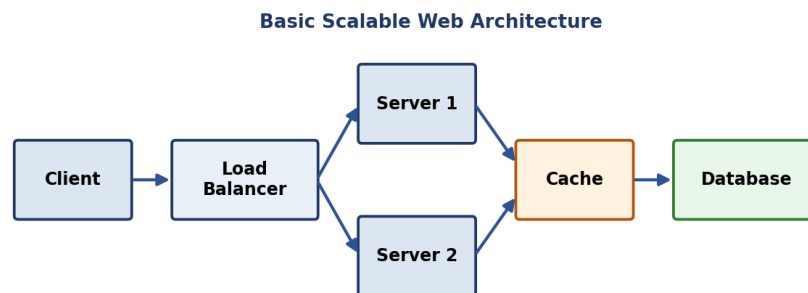
Answer:

For a high-traffic, read-heavy application, place a cache (e.g. Redis or Memcached) between the application servers and the database. On a read, check the cache first (a 'cache hit' returns data instantly without touching the database); on a miss, read from the database and populate the cache for next time. Choose a cache invalidation/expiration strategy (TTL, or explicit invalidation on writes) to balance freshness against cache-hit rate, and consider a cache-aside or write-through pattern depending on how tolerant the application is of slightly stale reads.

Q92. What is a load balancer, and what are common load-balancing strategies (round robin, least connections, etc.)?

Answer:

A load balancer distributes incoming client requests across multiple backend servers to prevent any single server from being overwhelmed and to improve availability if a server goes down. Common strategies: Round Robin (cycles requests evenly across servers), Least Connections (routes to the server currently handling the fewest active connections), Weighted Round Robin (accounts for servers with different capacities), and IP Hash (routes based on client IP for session stickiness).



Q93. What is idempotency, and why is it important when designing payment or retry-safe APIs?

Answer:

Idempotency means performing an operation multiple times has the same effect as performing it once. It's crucial for payment or retry-safe APIs because network failures often make clients unsure whether a request actually succeeded, causing them to retry - without idempotency, a retried payment request could charge a customer twice. It's typically implemented by having the client generate a unique idempotency key per logical operation, which the server stores and checks against on each retry to avoid re-processing the same request.

Q94. How would you design a rate limiter for an API?

Answer:

A common design uses the 'token bucket' or 'sliding window' algorithm: each client is allocated a bucket of tokens that refills at a fixed rate, and each request consumes a token - requests are rejected (or queued) once the bucket is empty, until it refills. This state (token counts and timestamps) is typically stored in a fast shared store like Redis so the rate limiter works consistently across multiple servers handling the same client's requests.

Q95. Explain the trade-offs between strong consistency and eventual consistency in a distributed system.

Answer:

Strong consistency guarantees that once a write completes, every subsequent read (from any replica) sees that write immediately - simpler to reason about, but often requires more coordination between nodes, increasing latency and reducing availability during network issues. Eventual consistency allows replicas to temporarily diverge after a write, guaranteeing only that all replicas will converge to the same value if no new writes occur for some time - offering better availability and lower latency, at the cost of applications occasionally reading stale data.

Q96. What is database sharding, and what challenges does it introduce (e.g., cross-shard queries)?

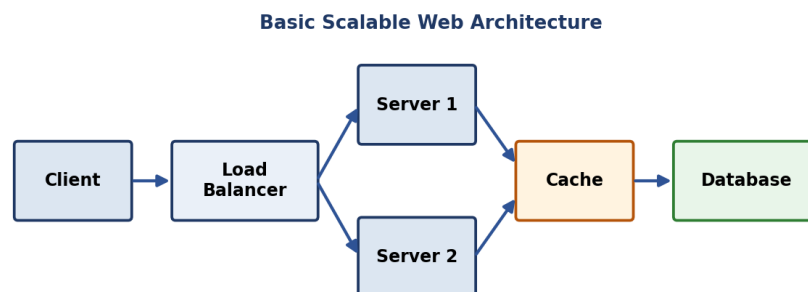
Answer:

Database sharding splits a large table's data horizontally across multiple database servers based on a shard key, so each shard holds only a subset of rows, improving write throughput and storage scalability beyond what one machine can handle. Challenges include queries or transactions that need data spanning multiple shards (requiring expensive cross-shard joins or scatter-gather queries), rebalancing data when adding/removing shards, and maintaining a consistent, well-distributed choice of shard key to avoid 'hot' shards.

Q97. How would you design a URL shortening service like bit.ly? Discuss the key components.

Answer:

Key components: a hash/encoding function (e.g. base62 encoding of an auto-incrementing ID, or a hash of the URL) to generate a short unique code; a database (often a fast key-value store) mapping short codes to original URLs; a redirect service that looks up the short code and issues an HTTP redirect; and a cache (e.g. Redis) in front of the database to serve frequently accessed short URLs quickly, since read traffic (redirects) vastly outnumbers write traffic (new URL creation).



Q98. Explain the CAP theorem and how it applies to designing a distributed database.

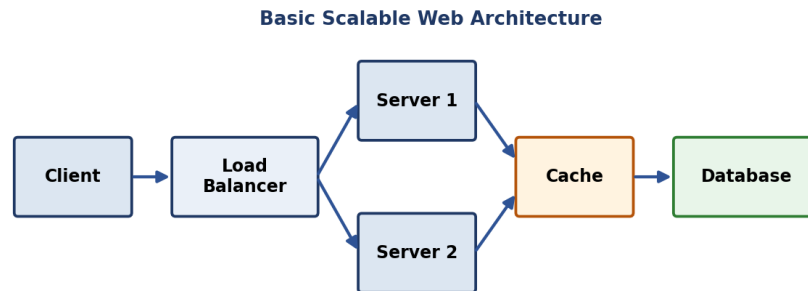
Answer:

The CAP theorem states that a distributed system can only guarantee two of the following three properties at the same time during a network partition: Consistency (every read gets the latest write), Availability (every request gets a non-error response), and Partition tolerance (the system keeps working despite network failures between nodes). Since partitions are essentially unavoidable in real distributed systems, designers effectively choose between prioritizing consistency (CP systems, e.g. many traditional databases in cluster mode) or availability (AP systems, e.g. many NoSQL databases) when a partition occurs.

Q99. What is the difference between horizontal and vertical scaling, and when would you use each?

Answer:

Horizontal scaling adds more machines/servers to distribute load across them (scale out), which is generally more resilient (no single point of failure) and has no hard upper limit, but requires the application to be designed to run across multiple nodes (e.g. stateless services, distributed data). Vertical scaling increases the resources (CPU/RAM) of a single existing machine (scale up), which is simpler with no architecture changes needed, but is limited by the maximum capacity of a single machine and doesn't improve fault tolerance.



Q100. Explain how you would design a notification system that can send millions of push notifications reliably.

Answer:

A reliable large-scale notification system typically uses a message queue (e.g. Kafka) to decouple the event that triggers a notification from the actual delivery: the triggering service publishes an event to the queue, and a pool of worker consumers pulls from the queue and calls the relevant push notification provider (e.g. FCM/APNs) for each recipient, retrying failed deliveries with backoff. This design absorbs traffic spikes, allows independent scaling of ingestion vs delivery workers, and avoids losing notifications if a downstream provider is temporarily slow or unavailable.

Note: 25 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.